

A2A Integration

ⓘ Note

This tutorial describes A2A integration in .NET apps; Python integration is in the works...

The Agent-to-Agent (A2A) protocol enables standardized communication between agents, allowing agents built with different frameworks and technologies to communicate seamlessly. The `Microsoft.Agents.AI.Hosting.A2A.AspNetCore` library provides ASP.NET Core integration for exposing your agents via the A2A protocol.

NuGet Packages:

- [Microsoft.Agents.AI.Hosting.A2A](#)
- [Microsoft.Agents.AI.Hosting.A2A.AspNetCore](#)

What is A2A?

A2A is a standardized protocol that supports:

- **Agent discovery** through agent cards
- **Message-based communication** between agents
- **Long-running agentic processes** via tasks
- **Cross-platform interoperability** between different agent frameworks

For more information, see the [A2A protocol specification](#).

Example

This minimal example shows how to expose an agent via A2A. The sample includes OpenAPI and Swagger dependencies to simplify testing.

1. Create an ASP.NET Core Web API project

Create a new ASP.NET Core Web API project or use an existing one.

2. Install required dependencies

Install the following packages:

.NET CLI

Run the following commands in your project directory to install the required NuGet packages:

Bash

```
# Hosting.A2A.AspNetCore for A2A protocol integration
dotnet add package Microsoft.Agents.AI.Hosting.A2A.AspNetCore --prerelease

# Libraries to connect to Azure OpenAI
dotnet add package Azure.AI.OpenAI --prerelease
dotnet add package Azure.Identity
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Extensions.AI.OpenAI --prerelease

# Swagger to test app
dotnet add package Microsoft.AspNetCore.OpenApi
dotnet add package Swashbuckle.AspNetCore
```

3. Configure Azure OpenAI connection

The application requires an Azure OpenAI connection. Configure the endpoint and deployment name using `dotnet user-secrets` or environment variables. You can also simply edit the `appsettings.json`, but that's not recommended for the apps deployed in production since some of the data can be considered to be secret.

User-Secrets

Bash

```
dotnet user-secrets set "AZURE_OPENAI_ENDPOINT" "https://<your-openai-  
resource>.openai.azure.com/"
dotnet user-secrets set "AZURE_OPENAI_DEPLOYMENT_NAME" "gpt-4o-mini"
```

4. Add the code to Program.cs

Replace the contents of `Program.cs` with the following code and run the application:

C#

```
using A2A.AspNetCore;
using Azure.AI.OpenAI;
using Azure.Identity;
using Microsoft.Agents.AI.Hosting;
using Microsoft.Extensions.AI;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOpenApi();
builder.Services.AddSwaggerGen();

string endpoint = builder.Configuration["AZURE_OPENAI_ENDPOINT"]
    ?? throw new InvalidOperationException("AZURE_OPENAI_ENDPOINT is not set.");
string deploymentName = builder.Configuration["AZURE_OPENAI_DEPLOYMENT_NAME"]
    ?? throw new InvalidOperationException("AZURE_OPENAI_DEPLOYMENT_NAME is not set.");

// Register the chat client
IChatClient chatClient = new AzureOpenAIClient(
    new Uri(endpoint),
    new DefaultAzureCredential())
    .GetChatClient(deploymentName)
    .AsIChatClient();
builder.Services.AddSingleton(chatClient);

// Register an agent
var pirateAgent = builder.AddAIAgent("pirate", instructions: "You are a pirate. Speak like a pirate.");

var app = builder.Build();

app.MapOpenApi();
app.UseSwagger();
app.UseSwaggerUI();

// Expose the agent via A2A protocol. You can also customize the agentCard
app.MapA2A(pirateAgent, path: "/a2a/pirate", agentCard: new()
{
    Name = "Pirate Agent",
    Description = "An agent that speaks like a pirate.",
    Version = "1.0"
});

app.Run();
```

Testing the Agent

Once the application is running, you can test the A2A agent using the following .http file or through Swagger UI.

The input format complies with the A2A specification. You can provide values for:

- `messageId` - A unique identifier for this specific message. You can create your own ID (e.g., a GUID) or set it to `null` to let the agent generate one automatically.
- `contextId` - The conversation identifier. Provide your own ID to start a new conversation or continue an existing one by reusing a previous `contextId`. The agent will maintain conversation history for the same `contextId`. Agent will generate one for you as well, if none is provided.

HTTP

```
# Send A2A request to the pirate agent
POST {{baseAddress}}/a2a/pirate/v1/message:stream
Content-Type: application/json
{
  "message": {
    "kind": "message",
    "role": "user",
    "parts": [
      {
        "kind": "text",
        "text": "Hey pirate! Tell me where have you been",
        "metadata": {}
      }
    ],
    "messageId": null,
    "contextId": "foo"
  }
}
```

Note: Replace `{{baseAddress}}` with your server endpoint.

This request returns the following JSON response:

JSON

```
{
  "kind": "message",
  "role": "agent",
  "parts": [
```

```
{
  "kind": "text",
  "text": "Arrr, ye scallywag! Ye'll have to tell me what yer after, or be I walkin' the plank? 🏴‍☠️"
},
{
  "messageId": "chatcmpl-CXtJbisgIJCg36Z44U16etngjAKRk",
  "contextId": "foo"
}
```

The response includes the `contextId` (conversation identifier), `messageId` (message identifier), and the actual content from the pirate agent.

AgentCard Configuration

The `AgentCard` provides metadata about your agent for discovery and integration:

C#

```
app.MapA2A(agent, "/a2a/my-agent", agentCard: new()
{
    Name = "My Agent",
    Description = "A helpful agent that assists with tasks.",
    Version = "1.0",
});
```

You can access the agent card by sending this request:

HTTP

```
# Send A2A request to the pirate agent
GET {{baseAddress}}/a2a/pirate/v1/card
```

Note: Replace `{{baseAddress}}` with your server endpoint.

AgentCard Properties

- **Name:** Display name of the agent
- **Description:** Brief description of the agent
- **Version:** Version string for the agent
- **Url:** Endpoint URL (automatically assigned if not specified)
- **Capabilities:** Optional metadata about streaming, push notifications, and other features

Exposing Multiple Agents

You can expose multiple agents in a single application, as long as their endpoints don't collide. Here's an example:

C#

```
var mathAgent = builder.AddAIAgent("math", instructions: "You are a math expert.");  
var scienceAgent = builder.AddAIAgent("science", instructions: "You are a science expert.");  
  
app.MapA2A(mathAgent, "/a2a/math");  
app.MapA2A(scienceAgent, "/a2a/science");
```

See Also

- [Hosting Overview](#)
- [OpenAI Integration](#)
- [A2A Protocol Specification](#)
- [Agent Discovery](#)

Last updated on 11/19/2025